# Latent-Function Video Diffusion: Generate Once, Query Videos at Any Continuous Control

Anonymous CVPR submission

Paper ID *****

## Abstract

*Video diffusion models support continuous control over video attributes, such as motion intensity and camera speed. Conventional models require a separate denoising process for each control value, even when the resulting videos share much of their visual content. We refer to this approach as* pointwise generation*. We introduce* ***Latent-Function Video Diffusion (LFVD)****, a framework that generates a shared latent representation of videos across a continuous control range using a single diffusion process. A lightweight query decoder then produces a complete video at any requested control value without additional denoising. LFVD learns this representation by encoding videos captured at different control values and training the query decoder to reconstruct their video latents. A pretrained video diffusion transformer subsequently learns to generate the shared representation using standard flow matching. Experiments on lighting and facial-expression control demonstrate that LFVD generates multiple videos with competitive visual quality using a single diffusion process. LFVD improves video quality and control consistency in lighting tasks, including at unseen control values, while maintaining comparable video quality in facial-expression generation. Although control consistency varies across tasks, LFVD reduces generation time when multiple control values are requested. These results highlight the potential of shared latent representations for efficient and continuous video generation.*

## 1. Introduction

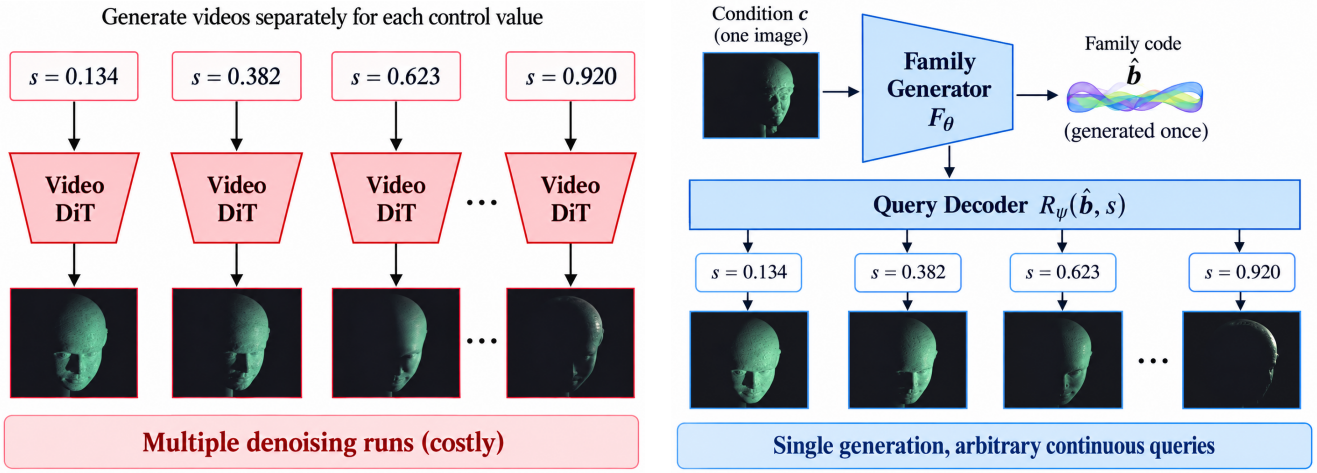
Video diffusion models have made substantial progress in generating videos from text and image conditions [2, 11, 12, 24]. They also support continuous control over video attributes such as motion intensity and camera speed. In interactive workflows, users often explore several control values before selecting an output. These requests may depict the same scene and subject while changing one attribute. The resulting videos share much of their visual content, yet generating each video separately repeats expensive computation for content that is largely unchanged. This motivates reusing information across control values.

Conventional video diffusion models generate each requested video separately. Given a control value $s$, the model runs a complete denoising process to produce the corresponding video. We refer to this approach as *pointwise generation*. Using the same initial noise can improve consistency across videos, but the model still performs a separate denoising process for each control value. Another practical approach generates videos at selected endpoints and interpolates their latents to obtain intermediate videos. This reduces repeated sampling, while the intermediate outputs follow a fixed interpolation rule.

We introduce **Latent-Function Video Diffusion (LFVD)**, a framework that generates videos across a continuous control range using a single diffusion process. LFVD generates a shared latent representation of videos at different control values. A lightweight query decoder takes this representation and a requested value $s \in [0, 1]$ to produce a complete video latent, which a frozen video VAE converts into video frames. The same representation supports multiple queries without additional denoising. Figure 1 illustrates this process.

LFVD learns the shared representation and its generation model in two stages. First, a frozen video VAE encodes videos observed at different control values. A family encoder combines their video latents into one representation, and a query decoder learns to reconstruct each video latent from the representation and its control value. We train these components using latent reconstruction loss. Next, we freeze the family encoder and query decoder and train a pretrained Video DiT to generate shared representations using standard flow matching. During inference, the Video DiT samples one representation from the input condition. The query decoder then produces videos at requested control values without requiring the original videos or further Video DiT evaluations.

We compare three ways to construct the shared representation. LFVD-Identity directly concatenates the ob-




(a) Pointwise generation: one diffusion process per control value.

(b) LFVD: one diffusion process, multiple video queries.

Figure 1. **Generate once, query many videos.** Pointwise generation runs a separate denoising process for each requested control value $s$. LFVD generates a shared latent representation once and uses a lightweight query decoder to produce videos at different control values. The thumbnails show held-out OpenIllumination frames. Multi-query costs are measured in Sec. 4.5.

served video latents. LFVD-Mix learns to combine information across latent time and control positions, while LFVD-Compact uses a shorter representation. All three use the same query decoder and diffusion training procedure. Their comparison helps identify how representation design affects generation quality and computational cost.

The shared representation contains more tokens than a single video latent, so its diffusion process has a higher initial cost. LFVD can offset this cost when multiple videos are requested. We measure the trade-off using synchronized inference time, analytical transformer FLOPs, and representation-size comparisons.

We evaluate LFVD on lighting and facial-expression control. OpenIllumination provides measured views of the same objects under various lights, which we arrange along a fixed camera path to form videos. MEAD provides facial videos at three ordered expression-intensity levels. We evaluate on unseen objects and actors and compare with pointwise generation and interpolation between endpoint video latents.

In lighting control, LFVD improves LPIPS from 0.4000 to 0.3216, PSNR from 19.29 to 21.14 dB, and control correlation from 0.786 to 0.933 compared with pointwise generation using shared initial noise. LFVD also generalizes to control values absent from training: with only three measured lighting positions for training, it improves LPIPS from 0.4103 to 0.3410 at six unseen positions with real measured targets. In facial-expression control, LFVD maintains video quality comparable to pointwise generation, although endpoint interpolation achieves stronger control ordering. Comparisons between LFVD-Identity and LFVD-Mix show that direct concatenation matches learned mixing overall and provides better fidelity on MEAD. Synchronized timing shows that LFVD becomes faster than batched pointwise generation after approximately seven lighting queries. Together, these results demonstrate the potential of shared latent representations for efficient video generation across continuous controls.

**Contributions.**

- We introduce LFVD, a framework that generates a shared latent representation with one diffusion process and uses a lightweight query decoder to produce complete videos at continuous control values.
- We develop a two-stage training procedure that learns a family encoder and query decoder, followed by training a Video DiT using standard flow matching.
- We evaluate LFVD on lighting and facial-expression control, demonstrating video quality, generalization to unseen controls, and multi-query efficiency through extensive experiments.

## 2. Related Work

### 2.1. Video Diffusion and Continuous Control

Video diffusion and continuous-control methods generate videos from visual conditions and allow users to adjust specific attributes. Early models developed spatiotemporal denoising and large-scale video synthesis [11, 12, 24]. Latent video diffusion operates on compressed representations, while Lumiere and Wan advance spatiotemporal modeling and pretrained video generation [1, 2, 27]. Motion-control methods guide generation using trajectories, motion fields, and regional constraints [9, 22, 29, 32]. Go-

with-the-Flow uses warped noise, and TokenMotion separates human and camera motion [3, 13]. MotionStone and Mojito control motion intensity, while Text Slider, FreeSliders, and TokenDial provide fine-grained attribute control [4, 7, 10, 18, 21].

### 2.2. Joint and Function-based Generation

Joint and function-based generation methods model related outputs and signals across multiple coordinates. Multi-view diffusion methods generate consistent images from different viewpoints: MVDream jointly models camera views, and SyncDreamer uses relationships among views conditioned on a reference image [17, 23]. Functional generative models represent signals over continuous input domains. Neural Diffusion Processes model distributions over functions [5]. Score-Based Diffusion Models in Function Space and Continuous-Time Functional Diffusion Processes extend diffusion to function-valued data [8, 14]. Functional Diffusion studies continuous signal generation, while HyperDiffusion generates neural-field parameters with diffusion [6, 30].

### 2.3. Efficient Diffusion Sampling

Efficient diffusion sampling methods reduce the computational cost of generating samples. DDIM introduces an alternative sampling procedure, while progressive distillation trains models to generate samples in fewer steps [20, 25]. Consistency Models and Latent Consistency Models learn consistency across noise levels to enable fast sampling [19, 26]. Other practical strategies for generating related outputs include reusing the same initial noise to improve consistency and interpolating the latents of generated endpoints to produce intermediate results. These approaches support efficient generation through reduced sampling steps, shared randomness, or interpolation.

## 3. Latent-Function Video Diffusion

LFVD generates a shared latent representation that supports videos at continuous control values. We first train a family encoder and a query decoder using videos observed at different controls. We then train a Video DiT to generate the shared representation. At inference, the same generated representation supports multiple video queries. Figure 2 illustrates the two training stages and inference process.

### 3.1. Conditional Videos and Latent Functions

Let $c$ denote the shared input condition, including a reference image and text, and let $s \in \mathcal{S} = [0, 1]$ be a continuous control value. We write $x_s$ for the video at control value $s$. A frozen video VAE consists of an encoder $\mathcal{E}_{\mathrm{VAE}}$ and a decoder $\mathcal{D}_{\mathrm{VAE}}$. We denote its video latent by $z_s = \mathcal{E}_{\mathrm{VAE}}(x_s)$. Let $C$, $T_z$, $H$, and $W$ denote the channel, temporal, height, and width dimensions of the *VAE latent*, respectively. Thus, $z_s \in \mathbb{R}^{C\times T_z\times H\times W}$; $H$ and $W$ refer to latent spatial dimensions, not the resolution of the original video.

For pointwise generation, let $G_{\theta_{\mathrm{PW}}}$ be a conditional diffusion sampler with parameters $\theta_{\mathrm{PW}}$. We use $\varepsilon$ for its initial Gaussian noise, $\hat{z}_s$ for the generated video latent, and $\hat{x}_s$ for the decoded video. Pointwise generation is

$$\hat{z}_s = G_{\theta_{\mathrm{PW}}}(c, s, \varepsilon), \qquad \hat{x}_s = \mathcal{D}_{\mathrm{VAE}}(\hat{z}_s). \tag{1}$$

The sampler performs a complete denoising process for each requested control value. LFVD instead uses a shared latent representation $\mathbf{b}$ and a query decoder $R_\psi$, parameterized by $\psi$, to reconstruct an individual video latent. Let $\mathcal{Z}_{\mathbf{b}}$ denote the resulting mapping from control values to video latents:

$$\begin{aligned} \mathcal{Z}_{\mathbf{b}} &: \mathcal{S} \to \mathbb{R}^{C\times T_z\times H\times W}, \\ \mathcal{Z}_{\mathbf{b}}(s) &= R_\psi(\mathbf{b}, s). \end{aligned} \tag{2}$$

A single representation therefore provides a video latent for each requested control value.

### 3.2. Learning a Family Representation

For each shared condition $c$, let $L$ denote the number of observed videos and let $\ell \in \{1, \dots, L\}$ index their control values $s_\ell$. We write $z_\ell = \mathcal{E}_{\mathrm{VAE}}(x_{s_\ell})$ for the corresponding latents and define the training family as $\mathcal{F}_c = \{(s_\ell, x_{s_\ell})\}_{\ell=1}^{L}$. We divide each video latent along the temporal dimension into a shared reference frame $z_{\mathrm{shared}} \in \mathbb{R}^{C\times 1\times H\times W}$ and the remaining $T_z - 1$ frames $z_{\ell,\mathrm{rest}}$, giving $z_\ell = [z_{\mathrm{shared}}, z_{\ell,\mathrm{rest}}]$. The total number of non-reference latent frames across the observed videos is $J = L(T_z - 1)$.

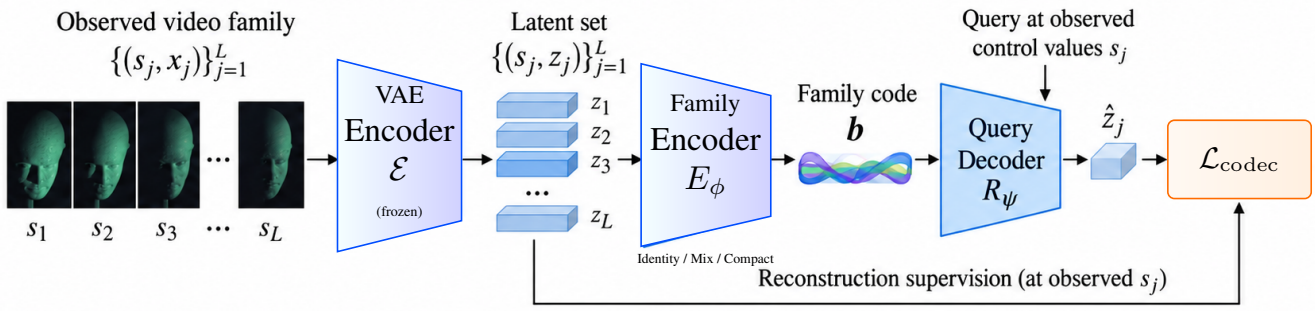
Let $E_\phi$ denote the family encoder, with parameters $\phi$ when learned. It combines the non-reference video latents into an encoded representation $\mathbf{b}_{0,\mathrm{rest}}$ containing $J'$ latent frames. Our encoder preserves the VAE latent channel and spatial dimensions $C$, $H$, and $W$, and transforms only the latent-time dimension. Therefore, $\mathbf{b}_{0,\mathrm{rest}} \in \mathbb{R}^{C\times J'\times H\times W}$. We concatenate this encoded representation with the unchanged reference frame to obtain the complete family code $\mathbf{b}_0$:

$$\begin{aligned} \mathbf{b}_{0,\mathrm{rest}} &= E_\phi\big(\{(s_\ell, z_{\ell,\mathrm{rest}})\}_{\ell=1}^{L}\big), \\ \mathbf{b}_0 &= [z_{\mathrm{shared}}, \mathbf{b}_{0,\mathrm{rest}}]. \end{aligned} \tag{3}$$

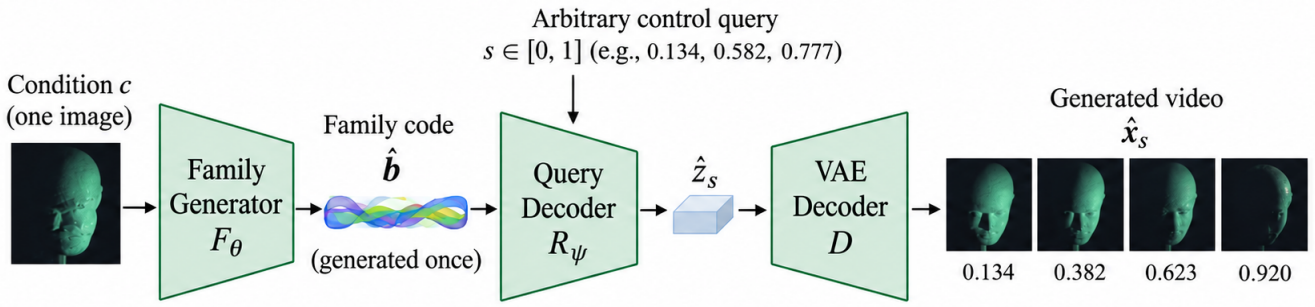
The family encoder can be fixed or learned.

**Family encoder variants.** We implement the family encoder $E_\phi$ in Eq. 3 using a temporal projection. Let $\mathrm{Emb}_\phi(s) \in \mathbb{R}^C$ denote an optional control embedding, $\lambda$ its scale, and $P_\phi \in \mathbb{R}^{J'\times J}$ a temporal projection. The embedding is broadcast over time and space, and the video latents are concatenated in control-value order:

$$\mathbf{b}_{0,\mathrm{rest}} = P_\phi\, \mathrm{Concat}_{\ell,\mathrm{time}}\left(z_{\ell,\mathrm{rest}} + \lambda\, \mathrm{Emb}_\phi(s_\ell)\right). \tag{4}$$




(a) Learning the family encoder and query decoder from observed videos.




(b) Generating one shared representation and querying multiple videos.

Figure 2. **LFVD training and inference.** (a) A frozen video VAE encodes the observed videos. The family encoder and query decoder learn a shared latent representation through reconstruction. We then freeze these components and train a Video DiT with flow matching. (b) At inference, the Video DiT samples the non-reference portion of one shared representation, while the reference latent remains fixed. The query decoder and frozen VAE produce videos at requested control values without further denoising.

The three encoder variants are defined by

$$\begin{aligned} \text{Identity:} \quad & J' = J,\ P_\phi = I,\ \lambda = 0, \\ \text{Mix:} \quad & J' = J,\ P_\phi \in \mathbb{R}^{J \times J}, \\ \text{Compact:} \quad & J' < J,\ P_\phi \in \mathbb{R}^{J' \times J}. \end{aligned}$$

Here, $I$ is the identity matrix. Identity directly concatenates the observed latents, Mix learns temporal mixing and control embeddings, and Compact reduces the representation length.

**Continuous query decoder.** Given the family code $\mathbf{b}_0$ and a control value $s$, the query decoder $R_\psi$ uses an $s$-conditioned neural network to generate a temporal projection $A_\psi(s)$ and a bias $\beta_\psi(s)$. Their dimensions and the decoding process are

$$\begin{aligned} A_\psi(s) &\in \mathbb{R}^{(T_z-1)\times J'}, \\ \beta_\psi(s) &\in \mathbb{R}^{C\times(T_z-1)}, \\ \hat{z}_{s,\text{rest}} &= A_\psi(s)\mathbf{b}_{0,\text{rest}} + \beta_\psi(s), \\ R_\psi(\mathbf{b}_0, s) &= [z_{\text{shared}}, \hat{z}_{s,\text{rest}}]. \end{aligned} \tag{5}$$

**Representation capacity.** Identity and Mix both use $J' = J$. On a fixed grid of observed control values, the query decoder can absorb an invertible temporal projection and the control-embedding offsets. These variants therefore have the same nominal reconstruction capacity when the Mix projection is invertible, although they can produce different code distributions for diffusion training. Compact reduces $J'$ and introduces a compression–fidelity trade-off. We compare these effects in Sec. 4.4.

### 3.3. Diffusion over the Family Code

After codec training, we freeze $E_\phi$ and $R_\psi$ and encode each training family into a clean code $\mathbf{b}_0$. Let $\theta$ denote the trainable Video DiT parameters. The model learns the conditional distribution of family codes using standard flow matching [15].

Let $t \in [0, 1]$ denote the normalized flow-matching noise level, and let $\varepsilon$ be a Gaussian-noise draw with the same shape as $\mathbf{b}_{0,\text{rest}}$. We denote the noisy family code by $\mathbf{b}_t$ and the target velocity of its non-reference portion by $\mathbf{v}_{\text{target}}$. The reference frame $z_{\text{shared}}$ remains fixed, while the non-reference portion is interpolated between its clean value and noise:

$$\begin{aligned} \mathbf{b}_t &= [z_{\text{shared}}, (1-t)\mathbf{b}_{0,\text{rest}} + t\varepsilon], \\ \mathbf{v}_{\text{target}} &= \varepsilon - \mathbf{b}_{0,\text{rest}}. \end{aligned} \tag{6}$$

Let $\mathbf{v}_{\theta,\text{rest}}$ denote the Video DiT's predicted velocity for the non-reference portion, and let $w(t)$ denote the loss weight

defined by the pretrained scheduler. We write $\mathcal{L}_{\mathrm{FM}}$ for the flow-matching loss and $\|\cdot\|_2^2$ for squared error over the predicted elements. The expectation covers training families, sampled noise levels, and Gaussian noise. We minimize

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}\left[w(t)\|\mathbf{v}_{\theta,\mathrm{rest}}(\mathbf{b}_t, t, c) - \mathbf{v}_{\mathrm{target}}\|_2^2\right]. \quad (7)$$

At inference, let $F_\theta$ denote the complete sampling process obtained by integrating the trained velocity predictor $\mathbf{v}_{\theta,\mathrm{rest}}$. We use $\varepsilon$ again for an independent Gaussian-noise draw, now with the shape of the family-code non-reference portion. The sampler keeps $z_{\mathrm{shared}}$ fixed and produces a family code $\hat{\mathbf{b}}$. Let $\hat{x}_s$ denote the video at the requested control $s$. Generation follows

$$\begin{aligned} \hat{\mathbf{b}} &= F_\theta(c, \varepsilon), \\ \hat{x}_s &= \mathcal{D}_{\mathrm{VAE}}\Big(R_\psi(\hat{\mathbf{b}}, s)\Big). \end{aligned} \quad (8)$$

The requested control value enters only the query decoder. Once the family code has been sampled, each additional video requires a query-decoder and VAE-decoder evaluation.

### 3.4. Multi-Query Generation Cost

The computational cost of LFVD depends on the family-code length $J'$ and the number of requested videos $M$. Let $\mathcal{C}_{\mathrm{diff}}(n)$ denote the cost of diffusion sampling with $n$ latent frames, keeping the backbone, spatial dimensions, and sampling steps fixed. Let $\mathcal{C}_D$ denote the cost of one VAE decoding and $\mathcal{C}_R(J')$ the cost of querying a family code of length $1 + J'$.

The query cost includes generating $A_\psi(s)$ and $\beta_\psi(s)$ and applying the temporal projection. Let $\mathcal{C}_{\mathrm{param}}(J')$ denote the cost of generating these projection parameters. Counting each multiply–accumulate as two operations, the query cost is approximately

$$\mathcal{C}_R(J') \approx \mathcal{C}_{\mathrm{param}}(J') + CHW(T_z - 1)(2J' + 1). \quad (9)$$

The total costs of pointwise generation and LFVD for $M$ videos are

$$\begin{aligned} \mathcal{C}_{\mathrm{PW}}(M) &= M\mathcal{C}_D + M\mathcal{C}_{\mathrm{diff}}(T_z), \\ \mathcal{C}_{\mathrm{LFVD}}(M) &= M\mathcal{C}_D + \mathcal{C}_{\mathrm{diff}}(1 + J') \\ &\quad + M\mathcal{C}_R(J'). \end{aligned} \quad (10)$$

Both methods require $M$ VAE decodings. The difference lies in the repeated diffusion cost of pointwise generation and the single diffusion cost followed by queries in LFVD.

Since the VAE costs cancel, LFVD becomes more efficient when

$$M > \frac{\mathcal{C}_{\mathrm{diff}}(1 + J')}{\mathcal{C}_{\mathrm{diff}}(T_z) - \mathcal{C}_R(J')}, \quad (11)$$

provided that $\mathcal{C}_{\mathrm{diff}}(T_z) > \mathcal{C}_R(J')$. This condition gives the minimum query count needed to amortize the family-generation cost under the sequential cost model. We evaluate the actual crossover, including batched pointwise generation, in Sec. 4.5.

## 4. Experiments

We evaluate video quality, continuous control, and multi-query cost. We also test unseen controls, encoder variants, and reconstruction error. Main results use held-out objects or actors, three seeds, and 50 denoising steps.

### 4.1. Experimental Setup

**Datasets.** *OpenIllumination* [16] captures real objects under one-light-at-a-time illumination. We fix a 17-frame camera path and vary nine measured light positions, normalized to $s \in [0, 1]$ by angular arc distance. Five object-disjoint folds contain 48/16/16 training, validation, and test families each, totaling 80 held-out families.

*MEAD* [28] provides facial videos at three ordinal expression intensities (low, medium, high). We align recordings using dynamic time warping on mel-frequency cepstral coefficients and their temporal derivatives, then form 17-frame families. Four actor-disjoint folds use five training, one validation, and two test actors each, totaling 112 held-out families. MEAD contains identifiable faces and voices. We use this existing dataset under its research terms and collect no new participant data; no new IRB review or consent process was conducted.

**Implementation.** We use the Wan2.2-TI2V-5B Video DiT and frozen video VAE [27] with the official DiffSynth trainer. All videos have 17 RGB frames at $256 \times 256$ resolution. We train the family encoder and query decoder for 8,000 updates, freeze them, and train rank-64 LoRA adapters on Video DiT attention and MLP projections for 3,456 updates. Pointwise models use the same backbone, optimizer, LoRA rank, number of updates, splits, and conditioning. Sampling uses 50 steps with seeds $\{100, 200, 300\}$.

**Baselines.** *Pointwise independent* samples each control value with independent Gaussian noise. *Pointwise shared-noise* uses the same initial noise for all requested values and runs a complete denoising process for each value. *Endpoint interpolation* generates the two endpoint videos and linearly interpolates their VAE latents. Unless specified otherwise, LFVD denotes LFVD-Mix. We also evaluate LFVD-Identity and LFVD-Compact.

**Metrics.** We report AlexNet LPIPS [31], PSNR, and SSIM against measured targets. Control correlation $\rho$ is the

Spearman correlation between $s$ and the target-projected response; constant outputs receive zero. Endpoint amplitude measures response range, and path second variation measures smoothness. We use paired object- or actor-level cluster bootstrap intervals (95%). Full settings and per-sample results are in the supplement.

### 4.2. Main Results

On OpenIllumination (Table 1), LFVD improves LPIPS from .4000 to .3216, PSNR from 19.29 to 21.14 dB, and correlation from .786 to .933 relative to shared-noise Pointwise. It uses one trajectory for nine values, compared with nine for Pointwise. LFVD's 95% object-level intervals are $[.3096, .3344]$ (LPIPS), $[20.58, 21.65]$ dB (PSNR), and $[.905, .960]$ (correlation). Shared noise yields higher Pointwise correlation than independent noise (.786 versus .468).

On MEAD, LFVD matches shared-noise Pointwise fidelity (.1151 versus .1157 LPIPS) but has lower correlation (.719 versus .743). Endpoint interpolation reaches .859 correlation, despite LFVD's larger endpoint response. These results show different fidelity and ordering behavior across control domains. Figures 4–6 show representative held-out videos, temporal consistency, and both queried control continua.

### 4.3. Generalization to Unseen Controls

We train Pointwise and LFVD-Mix using only three OpenIllumination positions, with indices $\{0, 4, 8\}$, and evaluate all nine measured positions. The six intermediate positions are excluded from both codec and diffusion training. We use the same five object-disjoint test folds, three seeds, and 50-step sampling. All targets are recorded measurements.

On unseen positions (Table 2), LPIPS improves from .4103 to .3410 and PSNR from 18.81 to 21.22 dB. Across 80 test families, 20 objects, and three seeds, paired gains are .0693 LPIPS (95% CI $[.0576, .0825]$), 2.41 dB PSNR ($[2.13, 2.69]$), and .0248 SSIM ($[.0191, .0305]$). Correlation over all nine values rises from .604 to .991, a gain of .386 ($[.316, .464]$).

### 4.4. Family Encoder Ablation

Identity and Mix are statistically tied on full-grid Lighting (LPIPS difference, Identity minus Mix: $-.0014$, 95% CI $[-.0058, .0030]$) and sparse unseen controls ($+.0007$, $[-.0015, .0028]$). On MEAD, Identity has lower LPIPS by .0082 ($[.0002, .0204]$) and higher PSNR by .19 dB ($[.00, .46]$). Thus the family-valued formulation is robust to the encoder choice, while learned mixing is not uniformly beneficial. The supplement reports complete encoder metrics and paired intervals for both datasets.

### 4.5. Measured Multi-Query Cost

As defined in Eq. 10, Pointwise repeats $\mathcal{C}_{\text{diff}}(T_z)$ for each requested video, while LFVD pays $\mathcal{C}_{\text{diff}}(1 + J')$ once and adds a query cost for each output. We compare both sequential Pointwise and batch-3 Pointwise, the strongest memory-feasible baseline. The supplementary timing curve and table report synchronized wall-clock time on one RTX 4090.

In Lighting, the measured crossover occurs at approximately seven queries: LFVD takes 15.40 s at $M = 7$, compared with 16.96 s for batch-3 Pointwise. At $M = 9, 17, 33$, LFVD is $1.23\times$, $1.97\times$, and $2.90\times$ faster than batched Pointwise. For MEAD's three control values, LFVD takes 5.74 s, compared with 6.60 s for batch-3 Pointwise and 11.47 s for sequential Pointwise.

The supplementary FLOP audit reports patch and text projections, attention projections and products, and feed-forward layers. Each multiply–accumulate is counted as two FLOPs. The diffusion-only crossover is 2.37 queries for MEAD and 6.82 for Lighting. At $M = 7$, Lighting Pointwise requires 1,214.47 TFLOPs, compared with 1,183.59 TFLOPs for LFVD; at $M = 9$, the ratio is $1.32\times$. These analytical results exclude query and VAE decoding and complement the measured timings. The sparse Lighting model uses 13 latent frames, 64.9% fewer than the 37-frame dense model. Its LPIPS over all nine queried values is .3271, compared with .3216 for the dense model, and its transformer-only crossover is 2.37 queries.

### 4.6. Ablations and Bottleneck Analysis

Exact-code decoding reaches .0642 LPIPS and perfect ordering, close to the frozen VAE floor of .0512; sampling the same representation raises LPIPS to .3365. Family-code diffusion, rather than the query decoder, is therefore the main quality bottleneck. Rank, training-family scaling, and coordinate ablations support the same conclusion. LFVD-Compact shortens the code from 37 to 28 frames and moves the analytical crossover from 6.82 to 5.09 queries, but degrades sampled LPIPS from .3300 to .4053. Complete protocols, tables, and qualitative decompositions are provided in the supplement.

### 4.7. Limitations

LFVD generates a larger latent representation than a single video and requires enough queries to amortize its initial cost. OpenIllumination provides controlled multi-view sequences; MEAD provides native videos but only three ordinal expression levels, where endpoint interpolation gives higher ordering correlation. The comparison between Identity and Mix is small and dataset-dependent, and the Compact variant loses fidelity. Finally, sampled-code quality remains below exact-code reconstruction, leaving family-code diffusion as the main limitation of the current model.

Table 1. **Results on held-out objects and actors.** Values are averaged over five OpenIllumination folds or four MEAD folds, with three seeds and 50 denoising steps. Trajectories and DiT calls are counted per video family. The best and second-best fidelity or ordering values are bold and underlined within each dataset.

| Dataset | Method | LPIPS↓ | PSNR↑ | SSIM↑ | $\rho$ ↑ | Traj. | DiT calls |
|---|---|---|---|---|---|---|---|
| Lighting | Pointwise independent | .3979 | 19.69 | .737 | .468 | 9 | 450 |
| | Pointwise shared-noise | .4000 | 19.29 | .731 | .786 | 9 | 450 |
| | Endpoint interpolation | .4059 | 19.33 | .727 | .860 | 2 | 100 |
| | LFVD | **.3216** | **21.14** | **.753** | **.933** | **1** | **50** |
| MEAD | Pointwise independent | **.1063** | **22.97** | **.764** | .577 | 3 | 150 |
| | Pointwise shared-noise | .1157 | 22.54 | .753 | .743 | 3 | 150 |
| | Endpoint interpolation | .1152 | 22.91 | .762 | **.859** | 2 | 100 |
| | LFVD | .1151 | **22.97** | .760 | .719 | **1** | **50** |

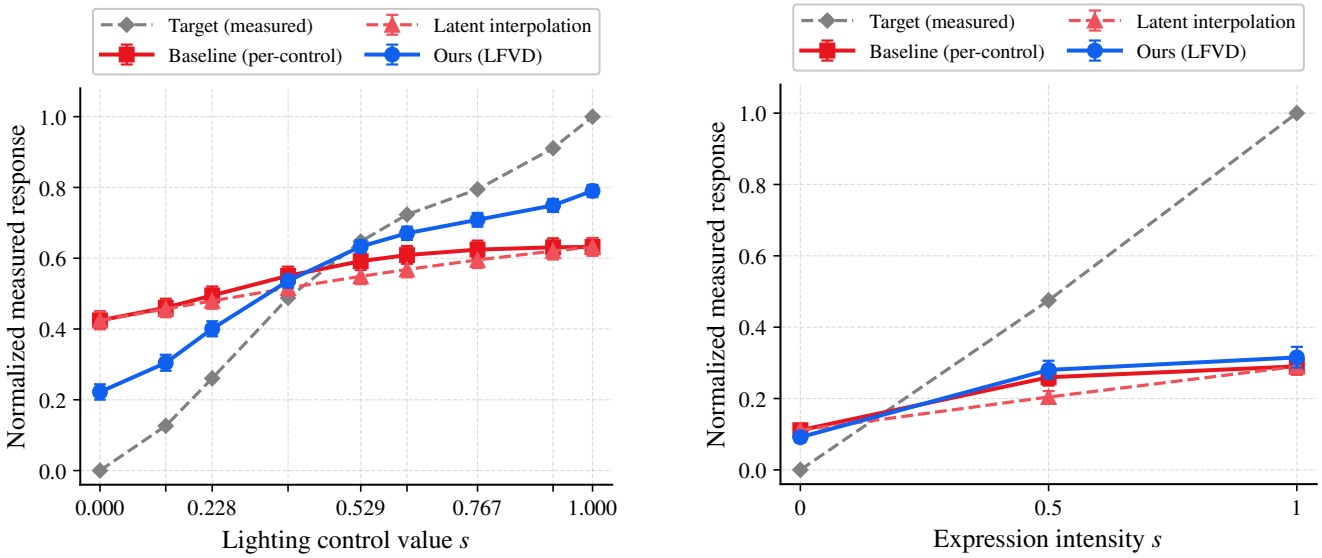



Figure 3. **Measured response over the requested control coordinate.** Curves aggregate the held-out Lighting and MEAD evaluations. LFVD queries the complete curve after one sampled family code; Pointwise executes a full trajectory for every coordinate.

Table 2. **Lighting control with three training positions.** Observed values are used in training; unseen values are the six remaining measured positions. Results cover five folds, three seeds, and 50 steps.

| Subset | Method | LPIPS↓ | PSNR↑ | SSIM↑ |
|---|---|---|---|---|
| Observed | Pointwise shared | .4062 | 18.48 | .7181 |
| | LFVD-Mix | **.2993** | **20.99** | **.7532** |
| Unseen | Pointwise shared | .4103 | 18.81 | .7232 |
| | LFVD-Mix | **.3410** | **21.22** | **.7480** |

## 5. Conclusion

LFVD samples a reusable family object for post-hoc queries. It improves Lighting quality and control, generalizes to locked-test intermediate positions, and amortizes over enough requests; MEAD preserves fidelity, not superior ordering. Identity matches Mix overall, locating the contribution in family-valued generation.

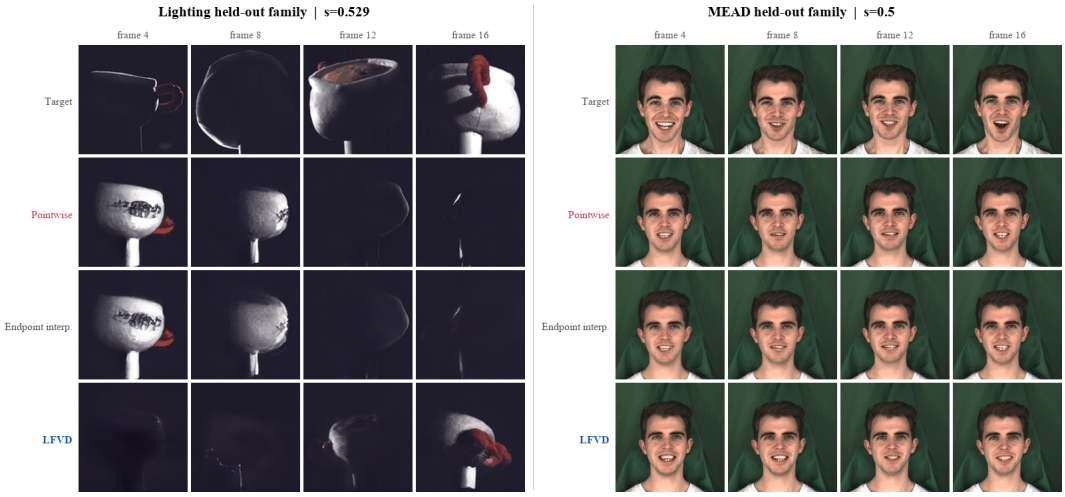



Figure 4. **Qualitative comparison at matched control values.** Columns show four generated frames from the same held-out family; rows compare the measured target, shared-noise Pointwise, endpoint latent interpolation, and LFVD. Lighting is queried at $s = .529$ and MEAD at $s = .5$. All panels use the locked 50-step evaluation and are shown without sharpening, alignment, or color correction.

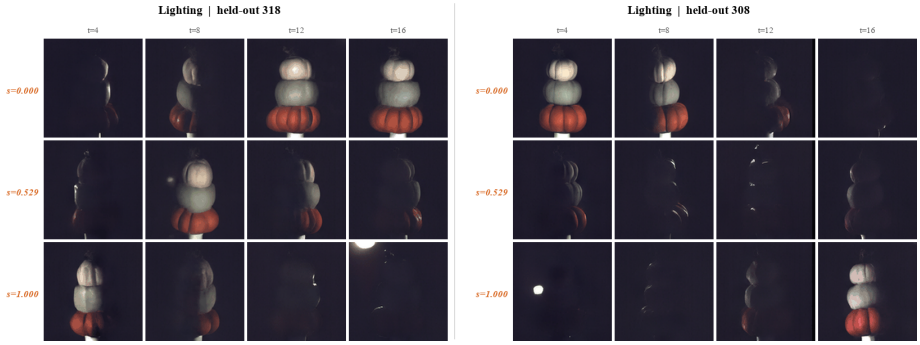



Figure 5. **Lighting intensity control from one LFVD trajectory.** Each panel shows a held-out object at three measured light coordinates; columns are temporal frames. All outputs are decoded from one sampled family code, with no query-time Video DiT call.

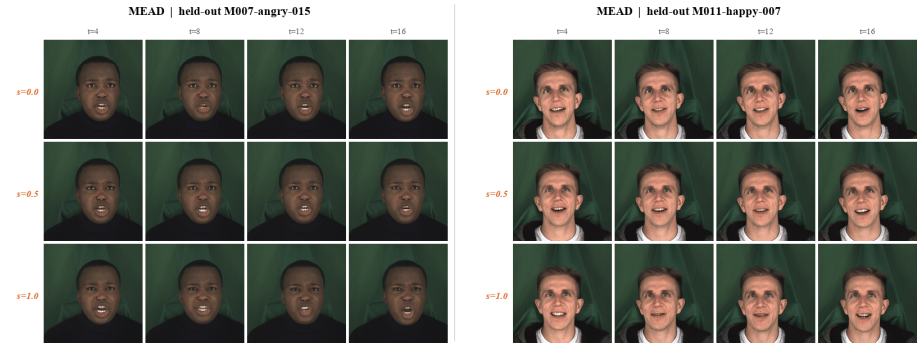



Figure 6. **Expression intensity control from one LFVD trajectory.** Each panel shows a held-out MEAD actor–utterance family queried at low, medium, and high measured intensity; columns are temporal frames.

## LLM Usage Statement

OpenAI Codex was used as an engineering and writing assistant for research ideation, implementation and debugging of experiment code, experiment automation, preparation of figures and tables, numerical-consistency checks, and prose editing. The human authors defined the research questions and methodology, selected the data and evaluation protocols, inspected the generated outputs, verified the reported results and citations, and retain full responsibility for the submission.

# References

[1] Omer Bar-Tal, Hila Chefer, Omer Tov, Charles Herrmann, Roni Paiss, Shiran Zada, Ariel Ephrat, Junhwa Hur, Guanghui Liu, Amit Raj, Yuanzhen Li, Michael Rubinstein, Tomer Michaeli, Oliver Wang, Deqing Sun, Tali Dekel, and Inbar Mosseri. Lumiere: A space-time diffusion model for video generation. *arXiv preprint arXiv:2401.12945*, 2024. 2

[2] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. In *CVPR*, pages 22563–22575, 2023. 1, 2

[3] Ryan Burgert, Yuancheng Xu, Wenqi Xian, Oliver Pilarski, Pascal Clausen, Mingming He, Li Ma, Yitong Deng, Lingxiao Li, Mohsen Mousavi, Michael Ryoo, Paul Debevec, and Ning Yu. Go-with-the-flow: Motion-controllable video diffusion models using real-time warped noise. In *CVPR*, pages 13–23, 2025. 3

[4] Pin-Yen Chiu, I-Sheng Fang, and Jun-Cheng Chen. Text slider: Efficient and plug-and-play continuous concept control for image/video synthesis via LoRA adapters. In *WACV*, 2026. 3

[5] Vincent Dutordoir, Alan Saul, Zoubin Ghahramani, and Fergus Simpson. Neural diffusion processes. In *ICML*, pages 8990–9012, 2023. 3

[6] Ziya Erkoç, Fangchang Ma, Qi Shan, Matthias Nießner, and Angela Dai. HyperDiffusion: Generating implicit neural fields with weight-space diffusion. In *ICCV*, 2023. 3

[7] Rotem Ezra, Hedi Zisling, Nimrod Berman, Ilan Naiman, Alexey Gorkor, Liran Nochumsohn, Eliya Nachmani, and Omri Azencot. FreeSliders: Training-free, modality-agnostic concept sliders for fine-grained diffusion control in images, audio, and video. *arXiv preprint arXiv:2511.00103*, 2025. 3

[8] Giulio Franzese, Giulio Corallo, Simone Rossi, Markus Heinonen, Maurizio Filippone, and Pietro Michiardi. Continuous-time functional diffusion processes. In *NeurIPS*, 2023. 3

[9] Daniel Geng, Charles Herrmann, Junhwa Hur, Forrester Cole, Serena Zhang, Tobias Pfaff, Tatiana Lopez-Guevara, Yusuf Aytar, Michael Rubinstein, Chen Sun, Oliver Wang, Andrew Owens, and Deqing Sun. Motion prompting: Controlling video generation with motion trajectories. In *CVPR*, pages 1–12, 2025. 2

[10] Xuehai He, Shuohang Wang, Jianwei Yang, Xiaoxia Wu, Yiping Wang, Kuan Wang, Zheng Zhan, Olatunji Ruwase, Yelong Shen, and Xin Eric Wang. Mojito: Motion trajectory and intensity control for video generation. *arXiv preprint arXiv:2412.08948*, 2024. 3

[11] Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P. Kingma, Ben Poole, Mohammad Norouzi, David J. Fleet, and Tim Salimans. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022. 1, 2

[12] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J. Fleet. Video diffusion models. In *NeurIPS*, pages 8633–8656, 2022. 1, 2

[13] Ruineng Li, Daitao Xing, Huiming Sun, Yuanzhou Ha, Jinglin Shen, and Chiuman Ho. TokenMotion: Decoupled motion control via token disentanglement for human-centric video generation. In *CVPR*, pages 1951–1961, 2025. 3

[14] Jae Hyun Lim, Nikola B. Kovachki, Ricardo Baptista, Christopher Beckham, Kamyar Azizzadenesheli, Jean Kossaifi, Vikram Voleti, Jiaming Song, Karsten Kreis, Jan Kautz, Christopher Pal, Arash Vahdat, and Anima Anandkumar. Score-based diffusion models in function space. *Journal of Machine Learning Research*, pages 1–62, 2025. 3

[15] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *ICLR*, 2023. 4

[16] Isabella Liu, Linghao Chen, Ziyang Fu, Liwen Wu, Haian Jin, Zhong Li, Chin Ming Ryan Wong, Yi Xu, Ravi Ramamoorthi, Zexiang Xu, and Hao Su. OpenIllumination: A multi-illumination dataset for inverse rendering evaluation on real objects. In *NeurIPS*, pages 36951–36962, 2023. 5

[17] Yuan Liu, Cheng Lin, Zijiao Zeng, Xiaoxiao Long, Lingjie Liu, Taku Komura, and Wenping Wang. SyncDreamer: Generating multiview-consistent images from a single-view image. In *ICLR*, 2024. 3

[18] Zhixuan Liu, Peter Schaldenbrand, Yijun Li, Long Mai, Aniruddha Mahapatra, Cusuh Ham, Jean Oh, and Jui-Hsien Wang. TokenDial: Continuous attribute control for text-to-video generation in visual dial space. *arXiv preprint arXiv:2603.27520*, 2026. 3

[19] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. *arXiv preprint arXiv:2310.04378*, 2023. 3

[20] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *ICLR*, 2022. 3

[21] Shuwei Shi, Biao Gong, Xi Chen, Dandan Zheng, Shuai Tan, Zizheng Yang, Yuyuan Li, Jingwen He, Kecheng Zheng, Jingdong Chen, Ming Yang, and Yinqiang Zheng. MotionStone: Decoupled motion intensity modulation with diffusion transformer for image-to-video generation. In *CVPR*, pages 22864–22874, 2025. 3

[22] Xiaoyu Shi, Zhaoyang Huang, Fu-Yun Wang, Weikang Bian, Dasong Li, Yi Zhang, Manyuan Zhang, Ka Chun Cheung, Simon See, Hongwei Qin, Jifeng Dai, and Hongsheng Li. Motion-I2V: Consistent and controllable image-to-video generation with explicit motion modeling. In *ACM SIGGRAPH Conference Papers*, pages 1–11, 2024. 2

[23] Yichun Shi, Peng Wang, Jianglong Ye, Long Mai, Kejie Li, and Xiao Yang. MVDream: Multi-view diffusion for 3d generation. In *ICLR*, 2024. 3

[24] Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, Devi Parikh, Sonal Gupta, and Yaniv Taigman. Make-A-Video: Text-to-video generation without text-video data. *arXiv preprint arXiv:2209.14792*, 2022. 1, 2

[25] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *ICLR*, 2021. 3

[26] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *ICML*, pages 32211–32252, 2023. 3

[27] Team Wan. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. 2, 5

[28] Kaisiyuan Wang, Qianyi Wu, Linsen Song, Zhuoqian Yang, Wayne Wu, Chen Qian, Ran He, Yu Qiao, and Chen Change Loy. MEAD: A large-scale audio-visual dataset for emotional talking-face generation. In *ECCV*, 2020. 5

[29] Shengming Yin, Chenfei Wu, Jian Liang, Jie Shi, Houqiang Li, Gong Ming, and Nan Duan. DragNUWA: Fine-grained control in video generation by integrating text, image, and trajectory. *arXiv preprint arXiv:2308.08089*, 2023. 2

[30] Biao Zhang and Peter Wonka. Functional diffusion. In *CVPR*, pages 4723–4732, 2024. 3

[31] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, pages 586–595, 2018. 5

[32] Zhongwei Zhang, Fuchen Long, Zhaofan Qiu, Yingwei Pan, Wu Liu, Ting Yao, and Tao Mei. MotionPro: A precise motion controller for image-to-video generation. In *CVPR*, pages 27957–27967, 2025. 2